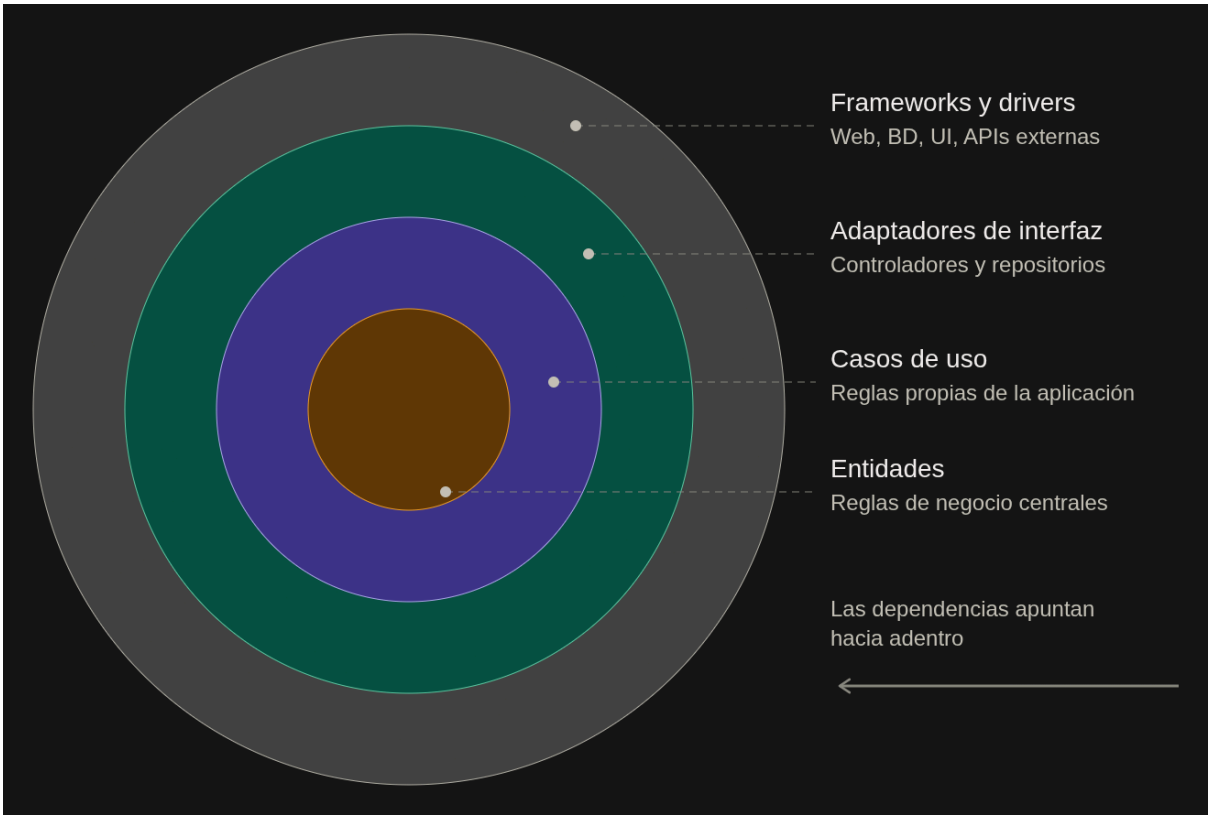
a) Investigación:

➢ **¿Qué significa que una arquitectura sea limpia “clean architecture”? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencia existe entre “clean architecture” y la arquitectura por capas tradicional? Mostrar con un gráfico la diferencia entre ambas.**

La **arquitectura limpia** (*Clean Architecture*), propuesta por Robert C. Martin (“Uncle Bob”), organiza el software en **capas concéntricas** donde lo más importante, las reglas de negocio, queda en el centro y es totalmente independiente de los detalles técnicos (base de datos, framework web, interfaz de usuario, librerías externas).

Su idea central es que **el negocio no debe depender de la tecnología**. Se puede cambiar de base de datos, de framework o de interfaz sin tocar las reglas que realmente dan valor a la aplicación.



Cómo funciona

Las capas, de adentro hacia afuera:

1. **Entidades:** los objetos y reglas de negocio más generales y estables (por ejemplo, “un pedido no puede tener total negativo”). No saben nada de bases de datos ni de pantallas.

2. **Casos de uso:** la lógica propia de la aplicación, es decir, lo que hace el sistema: “crear pedido”, “cancelar pedido”. Orquesta las entidades para cumplir cada objetivo.
3. **Adaptadores de interfaz:** convierten los datos entre el formato que usan los casos de uso y el que usa el exterior. Aquí viven los controladores, presentadores y la implementación de los repositorios.
4. **Frameworks y drivers:** los detalles tecnológicos: el framework web, la base de datos, la interfaz gráfica, servicios externos.

Ventajas

- **Independencia de frameworks:** el negocio no queda atado a Spring, Django o cualquier otro.
- **Independencia de la base de datos y de la UI:** se pueden cambiar sin reescribir las reglas.
- **Alta capacidad de prueba:** el núcleo se prueba sin servidor, base de datos ni interfaz, con pruebas rápidas.
- **Mantenibilidad:** cada parte tiene una responsabilidad clara y los cambios quedan localizados.
- **Longevidad:** las decisiones tecnológicas se pueden posponer o cambiar con el tiempo.
- **Facilita el trabajo en equipo:** hay límites claros entre las partes.
- **Se adapta bien a DDD y a microservicios:** cada servicio puede organizarse internamente de esta forma.

Desventajas

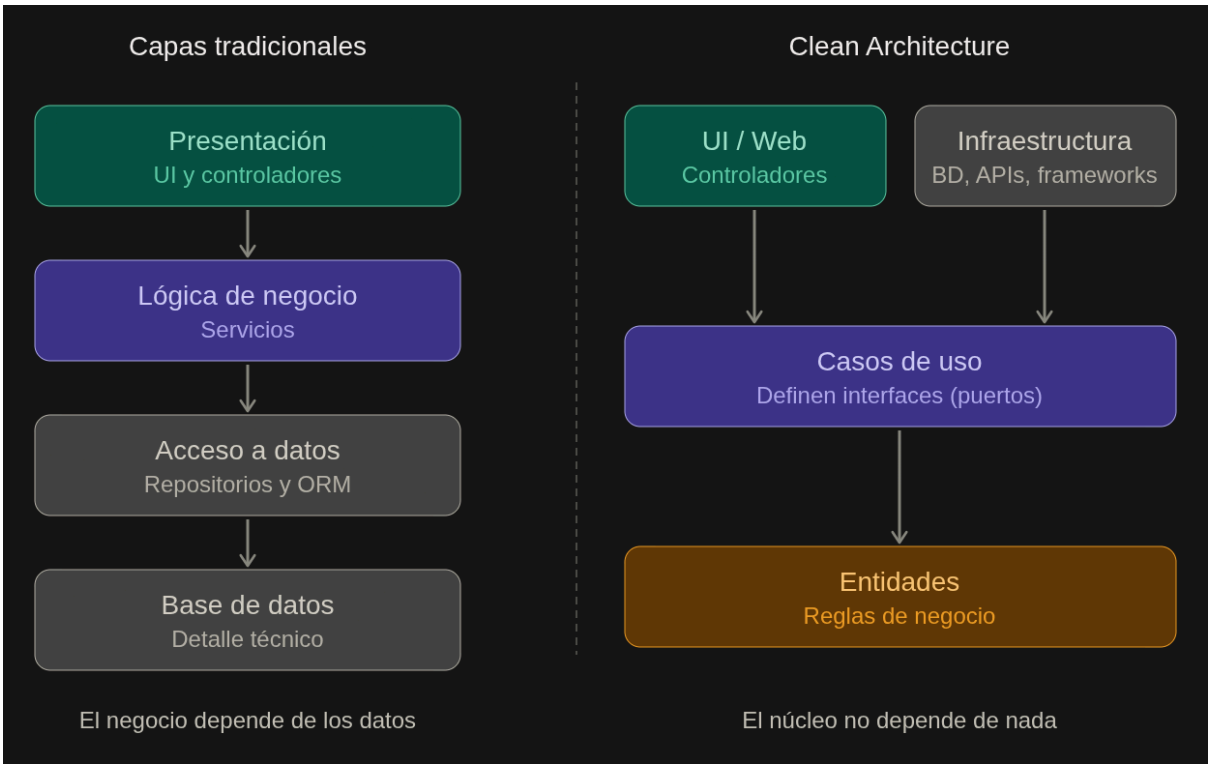
- **Más código y más estructura:** aparecen interfaces, mapeos entre objetos y muchas clases.
- **Curva de aprendizaje:** exige comprender bien la inversión de dependencias y la separación de responsabilidades.
- **Excesivo para proyectos pequeños:** en una aplicación simple tipo CRUD, la complejidad no se justifica.
- **Mapeos repetitivos:** convertir datos entre capas agrega trabajo y puede afectar levemente el rendimiento.
- **Riesgo de mal uso:** si se aplica sin criterio, se genera burocracia de código (clases que solo pasan datos de una capa a otra).
- **Desarrollo inicial más lento**, aunque suele compensarse a largo plazo.

Diferencia con la arquitectura por capas tradicional

La diferencia fundamental está en **hacia dónde apuntan las dependencias**.

En la arquitectura por capas tradicional, las dependencias van de arriba hacia abajo: la presentación depende de la lógica de negocio, y la lógica de negocio depende del acceso a datos y, en última instancia, de la base de datos. El negocio queda **atado a la tecnología de persistencia**.

En Clean Architecture, el núcleo no depende de nada. Las capas externas (interfaz de usuario y base de datos) son las que dependen de los casos de uso y de las entidades.

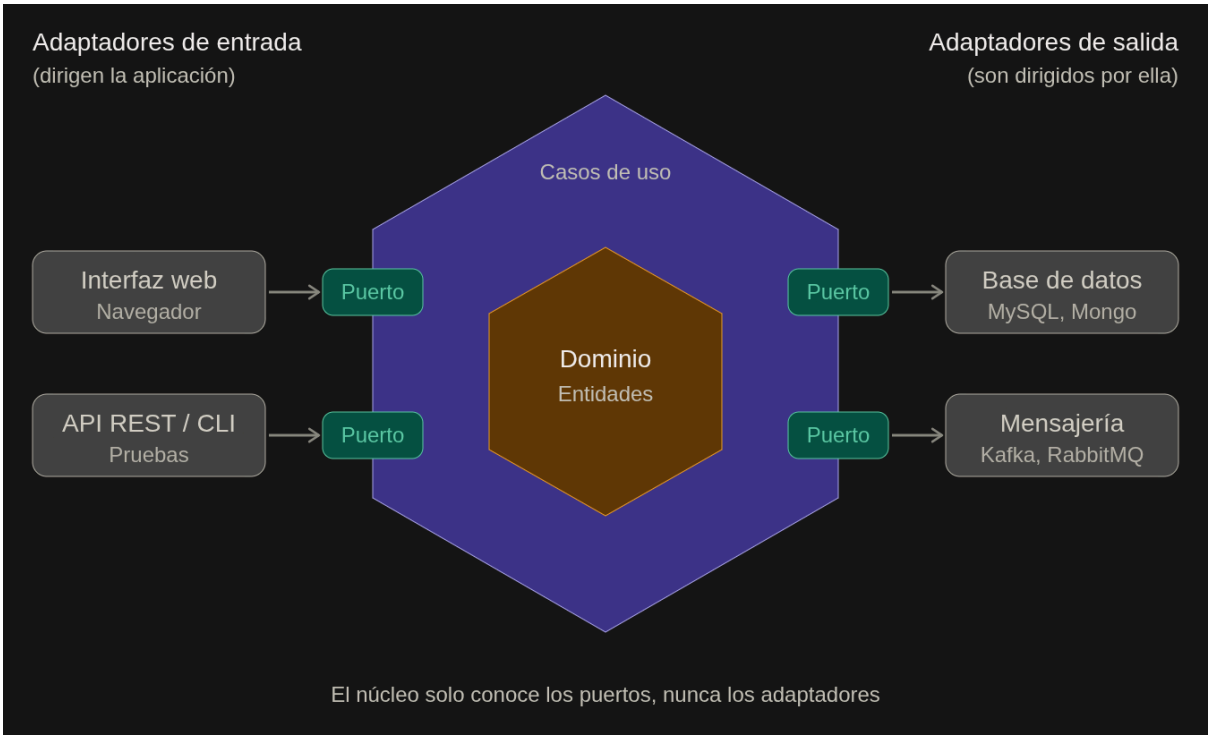


➢ ¿Qué es la arquitectura Hexagonal? Grafique la disposición de capas y explique su funcionamiento. Indique sus ventajas y desventajas. ¿Qué diferencias existen entre "clean architecture"?

La arquitectura Hexagonal, también llamada Puertos y Adaptadores (*Ports and Adapters*), fue propuesta por Alistair Cockburn en 2005. Su objetivo es aislar el núcleo de la aplicación (dominio y casos de uso) del mundo exterior, de modo que se pueda usar con distintos tipos de interfaz o de tecnología sin modificar la lógica de negocio.

El núcleo no sabe quién lo llama ni con qué tecnología guarda sus datos. Solo se comunica a través de puertos (interfaces), y el mundo exterior se conecta mediante adaptadores que traducen entre la tecnología concreta y esos puertos.

La forma de hexágono no tiene un significado técnico. Se eligió para mostrar que la aplicación puede tener varios lados de interacción y no solo el esquema "arriba y abajo" de las capas.



Cómo funciona

Los elementos:

1. **Núcleo (el hexágono):** contiene el dominio (entidades y reglas de negocio) y los casos de uso. No depende de ninguna tecnología.
2. **Puertos:** son interfaces definidas por el núcleo que describen cómo se comunica con el exterior. Son contratos, no código técnico.
3. **Adaptadores:** son las implementaciones concretas que conectan una tecnología real con un puerto. Traducen los datos entre ambos mundos.

Dos tipos de puertos y adaptadores:

- **Primarios o de entrada (*driving*, lado izquierdo):** quien llama a la aplicación. Ejemplos: un controlador REST, una pantalla web, una línea de comandos o una prueba automática. El puerto es la interfaz del caso de uso que ese adaptador invoca.
- **Secundarios o de salida (*driven*, lado derecho):** lo que la aplicación necesita llamar. Ejemplos: una base de datos, un servicio de correo, una cola de mensajes o una API externa. El núcleo define la interfaz (por ejemplo, `RepositorioPedidos`) y el adaptador la implementa.

Ventajas

- **Independencia tecnológica:** se cambia la base de datos, el framework o la interfaz sin modificar el negocio.

- **Alta capacidad de prueba:** se sustituyen los adaptadores reales por simulados (*mocks*) y se prueba el núcleo de forma rápida y aislada.
- **Múltiples puntos de entrada:** se agregan nuevas formas de acceso (API, CLI, mensajería) sin duplicar lógica.
- **Bajo acoplamiento:** el negocio queda protegido de los cambios tecnológicos.
- Código más mantenible: responsabilidades claras y separadas.
- **Posponer decisiones técnicas:** se puede empezar con una base de datos en memoria y decidir después.
- **Buena base para microservicios y para el diseño guiado por el dominio (DDD).**

Desventajas

- **Mayor cantidad de código:** interfaces, adaptadores y mapeos entre objetos agregan trabajo.
- **Curva de aprendizaje:** exige entender bien los puertos, la inversión de dependencias y el diseño del dominio.
- **Excesiva para aplicaciones simples:** en un CRUD pequeño, es complejidad innecesaria.
- **Riesgo de sobreingeniería:** crear puertos y adaptadores para todo, aunque nunca vayan a cambiar.
- **Mapeo constante de datos entre los modelos del núcleo y los de cada adaptador.**
- **Poca prescripción interna:** define cómo aislar el núcleo, pero no dice cómo organizarlo por dentro, por lo que cada equipo lo interpreta distinto.

Las 5 diferencias principales:

1. Qué explica

- **Hexagonal:** solo la frontera entre tu aplicación y el exterior.
- **Clean:** la frontera y el orden interno, capa por capa.

2. Cuántas partes tiene

- **Hexagonal:** 2 zonas, el núcleo (adentro) y los adaptadores (afuera).
- **Clean: 4 capas:** entidades, casos de uso, adaptadores de interfaz y frameworks.

3. Qué dice sobre el interior del núcleo

- **Hexagonal:** no dice nada, es “una caja” que cada equipo organiza como quiera.
- **Clean:** lo divide en entidades (reglas generales del negocio) y casos de uso (lo que hace la aplicación).

4. Cómo trata la entrada y la salida

- **Hexagonal:** distingue claramente lado de entrada (quien llama a la app: web, API) y lado de salida (lo que la app llama: base de datos, correo). Son dos tipos de puertos.
- **Clean:** no hace esa distinción. Todo lo externo es “adaptadores de interfaz”.

5. Qué regla destaca

- **Hexagonal:** “Todo lo externo se conecta mediante puertos y adaptadores”.
- **Clean**: “Las dependencias solo pueden apuntar hacia adentro” (la regla de dependencia).